

CY Tech - ING2 - PRF

Modélisation et vérification formelle d'un système bancaire distribué

Akka, Scala, réseaux de Pétri et LTL

Projet 2026 - Groupe 5

Souleim Mehdi Abbes Nahel

Projet	Système bancaire distribué critique : comptes, dépôts, retraits, virements et fermeture contrôlée.
Technologies	Scala 2.13.18, Akka Typed 2.8.5, sbt 1.12.5, analyseur maison de réseaux de Pétri, parseur LTL.
Périmètre analysé	<code>src/main/scala/banque</code> , <code>src/main/scala/petri</code> , tests Akka/Pétri/LTL, interface locale.
Propriétés ciblées	absence de deadlock, bornitude, vivacité, validité des transitions, cohérence des virements, solde jamais négatif.

2 mai 2026

Repository : <https://github.com/souleim95/Systeme-Bancaire-Distribue.git>

Table des matières

Résumé	3
1 Introduction	3
1.1 Contexte	3
1.2 Objectifs	3
1.3 Périmètre du rapport	3
2 État de l’art	4
2.1 Vérification formelle	4
2.2 Réseaux de Pétri	4
2.3 LTL	4
3 Concepts formels mobilisés	4
3.1 Système critique et abstraction	4
3.2 Éléments d’un réseau de Pétri	5
3.3 Espace d’états et graphe de réachabilité	5
3.4 Propriétés demandées	5
3.5 Rôle de LTL	6
4 Architecture fonctionnelle Akka/Scala	6
4.1 Vue générale	6
4.2 Hiérarchie des acteurs	7
4.3 Typage statique des protocoles	7
4.4 Flux de messages critiques	8
4.5 Règles métier	8
4.6 Machine à états d’un compte	8
5 Modèle formel par réseau de Pétri	9
5.1 Implémentation maison	9
5.2 Places et transitions	9
5.3 Schéma du réseau de virement	9
5.4 Places et marquage initial du virement	10
5.5 Transitions et correspondance avec le code	10
5.6 Espace d’états	10
6 Vérification des propriétés	11
6.1 Propriétés structurelles	11
6.2 Invariant métier principal	11
6.3 Invariants comportementaux	11
6.4 Détail des invariants critiques	12
6.5 Vérification LTL	13
7 Simulation et validation	13
7.1 Scénario Akka de référence	13
7.2 Comparaison Akka/Pétri	13
8 Interface de démonstration	14
9 Tests et commandes de validation	14
10 Traçabilité avec le cahier des charges	14

11 Limites et dette technique	15
12 Synthèse de vérification	15
13 Conclusion	16

Résumé

Ce projet porte sur la modélisation et la vérification d’une application distribuée critique : un système bancaire manipulant des comptes, des dépôts, des retraits et des virements. L’implémentation opérationnelle repose sur Akka Typed et Scala. Les comportements critiques sont ensuite abstraits dans un réseau de Pétri maison, sans outil externe, afin d’explorer l’espace d’états et de vérifier des propriétés de sûreté et de vivacité.

Le résultat obtenu couvre les exigences du cahier des charges : état de l’art, architecture acteur fonctionnelle, réseau de Pétri, analyse structurelle, invariants métier, model checking LTL, tests unitaires, interface de démonstration et comparaison entre simulation Akka et modèle formel.

1 Introduction

1.1 Contexte

Les systèmes bancaires distribués doivent traiter des opérations concurrentes tout en préservant des propriétés fortes : absence de perte d’argent, refus des opérations invalides, terminaison explicite des transactions et absence de blocage global. Dans ce contexte, une simple simulation ne suffit pas, car elle n’explore qu’un nombre limité de scénarios.

Le projet combine donc deux niveaux complémentaires :

- une application Akka/Scala qui simule le comportement réel du système ;
- un modèle formel par réseau de Pétri, vérifié structurellement et avec des formules LTL.

1.2 Objectifs

Les objectifs retenus sont les suivants :

1. étudier la vérification formelle et les réseaux de Pétri pour les systèmes critiques ;
2. concevoir une application distribuée bancaire sous forme d’acteurs Akka ;
3. identifier les flux de messages critiques, en particulier le virement ;
4. traduire les comportements critiques dans un réseau de Pétri maison ;
5. vérifier l’absence de deadlock, la bornitude, la vivacité et les invariants métier ;
6. comparer la simulation Akka et l’analyse formelle.

1.3 Périmètre du rapport

Le rapport vérifie le périmètre réellement livré dans le dépôt :

- l’architecture Akka Typed des acteurs `Banque` et `Compte` ;
- le protocole de messages défini dans `ProtocoleBancaire.scala` ;
- les opérations métier : création, dépôt, retrait, virement, consultation, historique et fermeture ;
- les réseaux de Pétri maison : compte simple, virement à deux comptes et réseau complet à trois comptes ;
- les tests unitaires Akka, les tests de propriétés Pétri et les tests LTL ;
- l’interface locale de démonstration qui expose les opérations et les résultats formels.

2 État de l'art

2.1 Vérification formelle

La vérification formelle consiste à représenter mathématiquement le comportement d'un système puis à vérifier automatiquement que certaines propriétés sont satisfaites. Elle est particulièrement adaptée aux systèmes critiques, car elle permet d'analyser des chemins d'exécution difficiles à reproduire par tests manuels.

On distingue principalement :

Sûreté une situation interdite ne doit jamais arriver. Exemple : un compte ne doit jamais avoir un solde négatif.

Vivacité une situation attendue finit par arriver. Exemple : une transaction initiée finit par être confirmée ou annulée.

Absence de deadlock

le système ne doit pas atteindre un état où aucune progression n'est possible.

2.2 Réseaux de Pétri

Un réseau de Pétri représente un système par des places, des transitions, des arcs et un marquage. Les places décrivent les états locaux, les transitions décrivent les événements, et les jetons décrivent l'état courant du système.

Ce modèle est adapté aux systèmes distribués car il exprime naturellement :

- la concurrence entre opérations indépendantes ;
- la causalité entre messages ;
- les conflits d'accès à une ressource partagée ;
- les états atteignables après chaque séquence d'exécution.

2.3 LTL

La logique temporelle linéaire (LTL) permet d'exprimer des propriétés sur l'évolution d'un système dans le temps. Les opérateurs principaux utilisés dans le projet sont :

Opérateur	Lecture	Exemple bancaire
Gphi	toujours ϕ	jamais de deadlock
Fphi	finalement ϕ	un verrou finit par être libéré
Xphi	au prochain état ϕ	l'état suivant est cohérent
phiUpsi	ϕ jusqu'à ψ	attente jusqu'à confirmation

3 Concepts formels mobilisés

Cette section explicite les notions demandées dans le cahier des charges et la manière dont elles sont utilisées dans le projet. L'objectif n'est pas seulement de citer les concepts, mais de montrer leur rôle dans la vérification du système bancaire distribué.

3.1 Système critique et abstraction

Un système est dit critique lorsqu'une erreur peut produire des conséquences importantes : perte d'argent, incohérence de données, blocage d'un service ou réponse incorrecte à un client. Dans notre cas, le virement bancaire est l'opération la plus sensible, car il modifie deux comptes et doit rester cohérent même lorsque plusieurs messages sont traités de façon concurrente.

L'abstraction consiste à ne conserver dans le modèle formel que les informations nécessaires à la vérification. Par exemple, le réseau de Pétri ne représente pas tous les montants possibles d'un compte. Il représente plutôt les états utiles à la sûreté : compte disponible, compte verrouillé, solde abstraitement valide, virement initié ou virement terminé. Cette simplification permet d'explorer l'espace d'états sans perdre les propriétés essentielles.

3.2 Éléments d'un réseau de Pétri

Un réseau de Pétri est défini par un ensemble de places, de transitions, d'arcs et par un marquage initial. Le tableau suivant relie chaque notion au cas bancaire :

Concept	Définition	Interprétation bancaire
Place	état local ou condition du système	compte disponible, compte verrouillé, virement en attente
Jeton	présence d'une ressource ou d'une condition vraie	un compte est libre, un solde est suffisant, une transaction est en cours
Transition	événement qui fait évoluer le système	dépôt, retrait, initiation d'un virement, confirmation ou annulation
Arc	relation de consommation ou de production entre place et transition	une opération consomme la disponibilité d'un compte puis produit un verrou
Marquage	répartition des jetons dans les places	état courant global du système bancaire abstrait

Une transition est franchissable lorsque toutes ses places d'entrée contiennent assez de jetons. Son franchissement consomme les jetons d'entrée et produit les jetons de sortie. Cette règle est importante pour l'invariant bancaire : le modèle ne peut pas consommer une ressource qui n'existe pas, ce qui correspond à l'idée qu'un retrait ou un virement ne doit pas être accepté sans condition de validité.

3.3 Espace d'états et graphe de réachabilité

L'espace d'états est l'ensemble de tous les marquages atteignables depuis le marquage initial. Dans le projet, il est construit automatiquement par parcours en largeur : à partir d'un marquage, on calcule toutes les transitions activées, puis tous les marquages obtenus après franchissement.

Le graphe de réachabilité est donc composé :

- de nœuds, qui sont les marquages atteignables ;
- d'arêtes, qui sont les transitions franchies entre deux marquages ;
- de chemins, qui correspondent aux séquences possibles d'exécution du système.

Cette exploration est plus forte qu'une simulation simple, car elle ne suit pas une seule exécution. Elle examine toutes les évolutions possibles du modèle, y compris les entrelacements concurrents entre opérations indépendantes.

3.4 Propriétés demandées

Les propriétés vérifiées dans ce projet correspondent directement aux attentes de l'énoncé :

Propriété	Question posée au modèle	Exemple dans le système bancaire
Validité des transitions	une transition n'est-elle exécutée que lorsque ses préconditions sont vraies ?	un virement ne démarre que si la source, la destination et le solde source sont disponibles
Absence de deadlock	existe-t-il un état atteignable où aucune transition ne peut s'exécuter ?	les comptes ne restent pas verrouillés définitivement
Cohérence des séquences	les chemins possibles respectent-ils l'ordre logique des opérations ?	un virement est initié avant d'être confirmé ou annulé
Bornitude	le nombre de jetons reste-t-il limité dans chaque place ?	le modèle ne crée pas une quantité non maîtrisée de ressources
Vivacité	une opération critique peut-elle finir par progresser ?	un verrou finit par être libéré
Invariant métier	une contrainte reste-t-elle vraie dans tous les états ?	un compte ne devient jamais négatif

3.5 Rôle de LTL

La logique LTL complète les propriétés structurelles du réseau de Pétri. Les propriétés structurelles observent le graphe d'états, tandis que LTL permet d'écrire des garanties sur les chemins d'exécution. Par exemple, $G!\text{deadlock}$ signifie que, sur toute exécution, aucun état bloqué n'est rencontré. La formule $G(\text{has_sourceLocked_p} \rightarrow F\text{has_sourceAvailable_p})$ exprime une propriété de vivacité : dès que la source est verrouillée, elle finit par redevenir disponible.

Ainsi, le projet relie trois niveaux : les règles métier du système bancaire, leur traduction en places et transitions, puis leur expression en propriétés vérifiables automatiquement.

4 Architecture fonctionnelle Akka/Scala

4.1 Vue générale

L'application est organisée autour de deux types d'acteurs principaux :

Acteur	Responsabilités
Banque	crée les comptes, garde la table des acteurs, route les opérations, agrège les soldes, arrête les comptes fermés.
Compte	porte l'état d'un compte : solde, historique, statut actif, virements en attente, validation des opérations.
ProtocoleBancaire	définit les commandes et les réponses échangées entre la banque, les comptes et le client.

Les fichiers principaux sont :

- `src/main/scala/banque/Banque.scala`
- `src/main/scala/banque/CompteBancaire.scala`
- `src/main/scala/banque/ProtocoleBancaire.scala`
- `src/main/scala/banque/FrontServer.scala`

4.2 Hiérarchie des acteurs

Le système repose sur une hiérarchie volontairement simple : un acteur **Banque** centralise la création des comptes et le routage, puis délègue l'exécution des opérations aux acteurs **Compte**. Cette organisation permet de séparer l'annuaire des comptes, maintenu par la banque, et l'état métier de chaque compte, maintenu localement par l'acteur concerné.

Élément	Rôle structurel	Garantie apportée
ActorSystem	racine d'exécution Akka Typed	isole le système bancaire dans une boucle de messages contrôlée
Banque	acteur d'orchestration et annuaire des comptes	évite l'accès direct non contrôlé aux comptes depuis le client
Compte(accountId)	acteur d'état pour un compte bancaire	séréalise les opérations concurrentes sur un même solde
Acteurs adaptateurs anonymes	relais temporaire de réponse vers la banque	rattache chaque réponse à la requête d'origine

La propriété essentielle est la sérialisation par acteur : deux messages envoyés au même compte ne modifient jamais simultanément le solde. Même si plusieurs clients envoient des opérations en parallèle, Akka les place dans la boîte aux lettres de l'acteur, puis **Compte** les traite un par un.

4.3 Typage statique des protocoles

Comme dans le rapport de référence, le typage des messages est une propriété structurelle importante. Le projet utilise Akka Typed : chaque acteur expose un **ActorRef[T]** qui n'accepte que les messages du type T. Les protocoles sont définis par des traits scellés, ce qui limite les commandes possibles et rend les échanges explicites.

Trait / type	Messages principaux	Destinataire
Banque . CommandeBanque	CreerCompteReq , OperationCompteBanque , ListerComptesReq , ConsulterSoldeGlobalReq , SupprimerCompteReq	acteur Banque uniquement
CommandeBancaire	Deposer , Retirer , Virement , ConsulterSolde , FermerCompte , ReceptionVirement , ConfirmationVirement	acteurs Compte
ReponseBancaire	OperationReussie , OperationEchouee , SoldeActuel , HistoriqueCompte , CompteCrÃ©Ã© , CompteFerme	client, banque ou adaptateur de réponse

Cette séparation évite par construction plusieurs erreurs de protocole : un client ne peut pas envoyer directement une commande de gestion interne à un compte sans passer par le type attendu, et la banque ne confond pas ses commandes d'orchestration avec les commandes métier d'un compte.

4.4 Flux de messages critiques

Les opérations simples suivent un routage direct :

```
Client -> Banque -> Compte -> Banque -> Client
```

Le virement est le flux le plus critique, car il implique deux comptes :

```
Client
-> Banque
-> Compte source
-> Compte destination
-> Compte source
-> Banque
-> Client
```

La source réserve immédiatement le montant avant de contacter le destinataire. Si le destinataire confirme, le virement est validé. Si le destinataire refuse ou répond de façon inattendue, la source rembourse le montant réservé. Ce mécanisme empêche un double débit concurrent et préserve la cohérence des fonds.

4.5 Règles métier

Les principales règles métier sont les suivantes :

- un compte ne peut pas être créé avec un solde initial négatif ;
- un dépôt doit avoir un montant strictement positif ;
- un retrait doit avoir un montant strictement positif et un solde suffisant ;
- un virement doit avoir un montant strictement positif, une source existante, une destination existante et un solde suffisant ;
- un compte fermé ne peut plus traiter d'opérations ;
- un compte avec un virement en attente ne peut pas être fermé.

4.6 Machine à états d'un compte

Dans le code, chaque compte possède un état interne `CompteState` composé de cinq informations : l'identifiant, le solde, l'historique, le booléen `estActif` et la table `pendingTransfers`. Cette structure correspond à une machine à états contrôlée exclusivement par l'acteur `Compte`.

État logique	Données portées	Transitions entrantes	Transitions sortantes
Actif libre	<code>estActif=true</code> , aucun virement en attente	création, fin de dépôt, fin de retrait, confirmation ou annulation de virement	dépôt, retrait, consultation, initiation de virement, fermeture
Virement réservé	solde source déjà débité, <code>pendingTransfers(txId)</code> présent	<code>Virement</code> accepté par la source	<code>ConfirmationVirement</code> confirmée ou échouée
Destination créée	historique de réception mis à jour	<code>ReceptionVirement</code> acceptée	réponse <code>OperationReussie</code> vers la source
Fermé	<code>estActif=false</code>	<code>FermerCompte</code> sans opération en cours	toutes les opérations métier sont refusées

L'acyclicité partielle de cette machine est importante : un compte fermé ne revient pas à l'état actif, et un virement réservé doit sortir de `pendingTransfers` par confirmation ou remboursement. Le refus de fermeture lorsque `pendingTransfers.nonEmpty` empêche de supprimer un compte pendant une opération critique.

5 Modèle formel par réseau de Pétri

5.1 Implémentation maison

Le modèle formel est implémenté dans `src/main/scala/petri`. Il n'utilise pas d'outil logiciel externe de réseau de Pétri, conformément au cahier des charges.

Fichier	Rôle
<code>PetriNet.scala</code>	structures <code>Place</code> , <code>Transition</code> , <code>Arc</code> , <code>Marking</code> , franchissement et graphe de réachabilité.
<code>BankingPetriNet.scala</code>	réseaux bancaires : compte simple, virement, réseau complet pour plusieurs comptes.
<code>PropertyChecker.scala</code>	vérification de deadlock, vivacité, bornitude, réversibilité et invariants.
<code>LTL.scala</code>	parseur, évaluateur et model checker LTL.
<code>Simulator.scala</code>	simulation interactive, aléatoire, par trace et par séquence.

5.2 Places et transitions

Pour un compte, le réseau abstrait les états suivants :

- compte disponible ;
- compte verrouillé pendant une opération ;
- validité abstraite du solde ;
- opération de dépôt, retrait ou libération.

Pour un virement, le réseau introduit :

- une source disponible ou verrouillée ;
- une destination disponible ou verrouillée ;
- une place de virement en attente ;
- une transition de confirmation ou de libération.

5.3 Schéma du réseau de virement

Le schéma suivant représente le cœur du réseau de Pétri utilisé pour modéliser un virement entre deux comptes. Les cercles sont des places, les rectangles sont des transitions et les flèches sont des arcs de consommation ou de production de jetons.

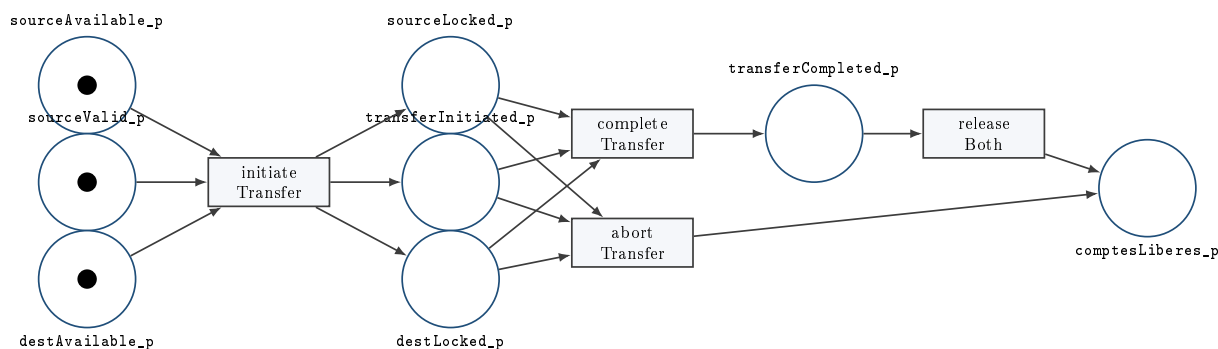


FIGURE 1 – Réseau de Pétri simplifié du virement bancaire

Le marquage initial contient trois jetons :

- un jeton dans `sourceAvailable_p`, qui indique que le compte source est libre ;

- un jeton dans `sourceValid_p`, qui indique que le solde source est suffisant ;
- un jeton dans `destAvailable_p`, qui indique que le compte destination est libre.

La transition `initiateTransfer_t` ne peut donc être franchie que si ces trois conditions sont vraies en même temps. Après franchissement, les deux comptes sont verrouillés et un jeton est placé dans `transferInitiated_p`.

Deux suites sont alors possibles. La transition `completeTransfer_t` représente le cas nominal : le virement est confirmé puis `releaseBothAccounts_t` rend les comptes disponibles. La transition `abortTransfer_t` représente l'échec contrôlé : le virement est annulé et les comptes sont également libérés. Dans les deux cas, les verrous ne restent pas bloqués, ce qui soutient les propriétés d'absence de deadlock et de vivacité.

5.4 Places et marquage initial du virement

La table suivante reprend le niveau de détail du rapport de référence : chaque place du réseau de virement reçoit une signification métier et un marquage initial. Le modèle est volontairement abstrait : il ne stocke pas le montant exact, mais uniquement les conditions nécessaires à la validité du virement.

Place	Rôle sémantique	Marquage initial
<code>sourceAvailable_p</code>	le compte source peut accepter une opération	1 jeton
<code>sourceValid_p</code>	le solde source est abstraitement suffisant pour débiter	1 jeton
<code>sourceLocked_p</code>	la source est réservée pendant le virement	0 jeton
<code>destAvailable_p</code>	le compte destination peut recevoir le virement	1 jeton
<code>destLocked_p</code>	la destination est réservée pendant le virement	0 jeton
<code>transferInitiated_p</code>	le virement a été lancé mais pas encore tranché	0 jeton
<code>transferCompleted_p</code>	le virement a été confirmé et attend la libération	0 jeton

5.5 Transitions et correspondance avec le code

Transition	Préconditions et effets	Correspondance Scala/Akka
<code>initiateTransfer_t</code>	consomme la disponibilité source, la validité source et la disponibilité destination ; produit les verrous source/destination et l'état <code>transferInitiated_p</code>	réception de <code>Virement</code> , validation du montant, débit réservé et envoi de <code>ReceptionVirement</code>
<code>completeTransfer_t</code>	consomme le virement initié et les deux verrous ; produit <code>transferCompleted_p</code>	réception d'une <code>OperationReussie</code> depuis la destination dans <code>ConfirmationVirement</code>
<code>abortTransfer_t</code>	consomme le virement initié et les deux verrous ; restaure la disponibilité source, la destination et la validité source	réponse <code>OperationEchouee</code> ou réponse inattendue de la destination, puis remboursement de la source
<code>releaseBothAccounts_t</code>	consomme <code>transferCompleted_p</code> ; rend les comptes disponibles et restaure la validité source	suppression de <code>pendingTransfers(txId)</code> après confirmation et retour vers un état libre

Cette correspondance justifie le modèle : chaque transition importante du réseau représente une étape atomique visible dans le protocole Akka. Les détails non essentiels à la vérification, comme le texte exact des messages ou le contenu complet de l'historique, sont abstraits.

5.6 Espace d'états

Le graphe de réachabilité est construit par parcours en largeur. L'implémentation se trouve dans la méthode `getReachabilityGraph` de la classe `PetriNet`. Chaque marquage atteint est exploré, puis les transitions activées produisent les marquages suivants.

Les résultats observés pour les réseaux fournis sont :

Réseau	Places	Transitions	Arcs	États atteignables
Compte simple	4	3	6	2
Virement deux comptes	7	4	22	3
Réseau complet trois comptes	15	21	96	216

6 Vérification des propriétés

6.1 Propriétés structurelles

Le vérificateur `PropertyChecker` vérifie les propriétés suivantes :

Absence de deadlock

aucun marquage atteignable ne doit être terminal.

Vivacité depuis les états atteignables, les transitions critiques peuvent redevenir exécutables.

Bornitude le nombre de jetons par place reste borné.

Réversibilité le modèle peut revenir au marquage initial lorsque la structure le garantit.

Invariant les marquages ne produisent jamais de valeur négative.

Sur les réseaux du projet, les analyses attendues sont en succès :

Propriété	Résultat	Interprétation
Absence de deadlock	PASS	le système peut toujours progresser
Vivacité	PASS	les opérations critiques ne restent pas définitivement bloquées
Bornitude	PASS	pas d'explosion de jetons dans les places analysées
Marquages non négatifs	PASS	cohérent avec l'invariant métier principal

6.2 Invariant métier principal

L'invariant principal est :

Un compte bancaire ne doit jamais avoir un solde négatif.

Dans l'application Akka, cet invariant est garanti par les gardes de validation des acteurs **Banque** et **Compte**. Dans le réseau de Pétri, il est représenté par une abstraction en entiers naturels : une transition ne peut être franchie que si les places d'entrée possèdent assez de jetons, et l'opération `decrement` ne produit pas de marquage négatif.

6.3 Invariants comportementaux

Les invariants suivants croisent trois sources : la lecture du code Akka, les tests unitaires et le réseau de Pétri. La colonne *niveau* indique la force de la garantie dans le périmètre du projet.

#	Invariant	Mécanisme dans le code	Vérification Pétri/LTL	Niveau
I-1	Solde jamais négatif	gardes <code>soldeInitial >= 0</code> , <code>montant > 0</code> , <code>solde >= montant</code> avant retrait ou virement	consommation de jetons seulement si les places d'entrée sont marquées; <code>Marking.decrement</code> ne crée pas de valeur négative	Fort
I-2	Unicité d'un compte	<code>BanqueState.comptes.contains(accountId)</code> bloque une création en doublon	les identifiants concrets sont abstraits; le réseau complet crée une place par compte connu	Fort côté code

#	Invariant	Mécanisme dans le code	Vérification Pétri/LTL	Niveau
I-3	Opérations interdites sur compte fermé	<code>estActif</code> est vérifié avant dépôt, retrait, virement et consultation	l'absence de jeton de disponibilité bloque les transitions d'opération	Fort
I-4	Pas de double débit concurrent	l'acteur <code>Compte</code> traite les messages séquentiellement et réserve immédiatement le montant source	<code>sourceAvailable_p</code> est consommée par <code>initiateTransfer_t</code> , puis la source reste verrouillée	Fort
I-5	Virement compensé en cas d'échec	<code>ConfirmationVirement</code> rembourse la source si la destination répond par échec ou réponse inattendue	<code>abortTransfer_t</code> restaure <code>sourceAvailable_p</code> , <code>destAvailable_p</code> et <code>sourceValid_p</code>	Fort
I-6	Fermeture impossible pendant un virement	<code>FermerCompte</code> refuse si <code>pendingTransfers.nonEmpty</code>	le modèle représente l'opération en cours par des verrous et une place <code>transferInitiated_p</code>	Fort
I-7	Libération des verrous	confirmation ou annulation retire le virement des opérations en attente	$G(\text{has_sourceLocked_p} \rightarrow \text{Fhas_sourceAvailable_p})$ vérifie que la source verrouillée finit par redevenir disponible	Fort formel
I-8	Cohérence du total bancaire	la banque met à jour son cache de soldes après <code>OperationReussie</code> et après <code>ForwardTransferResponse</code>	le réseau abstrait la conservation des ressources, pas les montants exacts	Moyen
I-9	Graphe d'états fini et borné	nombre fini de comptes et de commandes modélisées	<code>checkBoundedness</code> et exploration BFS : 216 états pour trois comptes	Fort formel
I-10	Absence de deadlock	les opérations normales ont toujours une réponse de succès ou d'échec	<code>checkNoDeadlock</code> et formules <code>Genabled/G!deadlock</code>	Fort formel

6.4 Détail des invariants critiques

I-1 et I-4 – sûreté du débit. Le débit est le point le plus risqué : une erreur pourrait faire passer un compte sous zéro ou appliquer deux fois le même retrait. Dans le code, le retrait et le virement vérifient d'abord que le montant est positif et que `state.solde >= montant`. Ensuite, le virement réserve immédiatement le montant côté source, avant de contacter la destination. Comme un acteur Akka traite ses messages séquentiellement, une seconde opération concurrente verra déjà le solde réservé et ne pourra pas réutiliser les mêmes fonds.

I-5 – atomicité par compensation. Le système ne réalise pas une transaction distribuée ACID complète, mais il implémente une compensation explicite. Si le destinataire refuse le virement parce que le compte est fermé, ou si la réponse reçue n'est pas une confirmation attendue, la source ajoute de nouveau le montant réservé à son solde et supprime l'entrée `pendingTransfers`. Cette logique correspond à la branche `abortTransfer_t` du réseau de Pétri.

I-7 – vivacité des virements. Dans le modèle formel, tout verrou finit par être libéré, soit par confirmation, soit par annulation. Dans le code actuel, cette propriété est vérifiée pour les comptes qui répondent normalement. Une panne silencieuse d'un acteur destination sans réponse explicite n'est pas encore couverte par un timeout applicatif ; c'est donc une limite technique à surveiller hors scénario nominal.

I-8 – cohérence banque/comptes. La banque maintient un cache `soldes` pour lister les comptes et calculer le solde global. Ce cache est mis à jour à partir des réponses des comptes. Les tests vérifient la cohérence après dépôt, retrait et virement. Le réseau de Pétri, lui, ne représente pas les montants précis ; il vérifie la cohérence structurelle du flux, pas l'arithmétique complète des soldes.

6.5 Vérification LTL

Le projet implémente un parseur et évaluateur LTL dans `LTL.scala`. Les opérateurs supportés incluent `!`, `\&`, `|`, `->`, `X`, `F`, `G`, `U` et `R`.

Quelques formules utiles :

Formule	Signification
<code>Genabled</code>	globalement, au moins une transition est disponible.
<code>G!deadlock</code>	aucun état atteignable n'est un deadlock.
<code>G(has_sourceLocked_p->Fhas_sourceAvailable_p)</code>	une source verrouillée finit par être libérée.
<code>G(has_transferInitiated_p->F(has_transferCompleted_p has_sourceAvailable_p))</code>	un virement initié finit par être confirmé ou annulé proprement.

Les deadlocks sont représentés par une boucle stutter sur l'état bloqué, ce qui permet de vérifier les propriétés temporelles sur des chemins infinis.

7 Simulation et validation

7.1 Scénario Akka de référence

Le scénario de démonstration crée trois comptes :

Compte	Solde initial
ACC-001	1000 EUR
ACC-002	500 EUR
ACC-003	750 EUR

Puis il exécute :

1. dépôt de 200 EUR sur ACC-001 ;
2. retrait de 100 EUR sur ACC-002 ;
3. virement de 300 EUR de ACC-001 vers ACC-002.

Les soldes attendus sont :

Compte	Solde final attendu
ACC-001	900 EUR
ACC-002	700 EUR
ACC-003	750 EUR

7.2 Comparaison Akka/Pétri

La simulation Akka montre le comportement opérationnel réel : messages, acteurs, réponses, historiques et soldes. Le réseau de Pétri montre l'abstraction formelle : disponibilité, verrouillage, transitions critiques, états atteignables et absence de blocage.

Opération	Côté Akka	Côté Pétri
Dépôt	augmentation du solde et historique	transition de dépôt puis libération du compte
Retrait	garde <code>solde >= montant</code>	transition franchissable seulement si le compte est valide
Virement	réservation source, crédit destination, confirmation ou remboursement	verrouillage source/destination, place en attente, libération

8 Interface de démonstration

Une interface web locale permet de présenter le projet sans passer uniquement par la console. Elle est servie par `FrontServer.scala`.

```
sbt "runMain banque.FrontServer 8080"
```

Puis ouvrir :

```
http://localhost:8080
```

L'interface permet de :

- créer, fermer et consulter des comptes ;
- effectuer dépôts, retraits et virements ;
- vérifier le total de la banque ;
- afficher l'analyse formelle du réseau de Pétri ;
- tester des formules LTL comme `Genabled` ou `G!deadlock`.

9 Tests et commandes de validation

Les tests unitaires couvrent les comportements bancaires, les réseaux de Pétri et LTL :

- `src/test/scala/banque/CompteSpec.scala`
- `src/test/scala/banque/BanqueSpec.scala`
- `src/test/scala/banque/PetriNetBankingTests.scala`
- `src/test/scala/banque/LTLTests.scala`

Commandes principales :

```
sbt clean test
sbt "runMain banque.PetriNetIntegrationDemo"
sbt "runMain banque.LTLIntegrationExample"
sbt "runMain banque.FrontServer 8080"
```

10 Traçabilité avec le cahier des charges

Attendu	Couverture dans le projet
Sources bibliographiques	Section état de l'art et fichier <code>docs/etat_de_l_art.md</code> .
Modèle Akka/Scala fonctionnel	Acteurs <code>Banque</code> et <code>Compte</code> , protocole typé, tests et interface.

Attendu	Couverture dans le projet
Réseau de Pétri	Implémentation maison dans <code>src/main/scala/petri</code> , avec exploration de l'espace d'états.
Rapport de vérification	Ce rapport, <code>docs/rapport_verification.md</code> , guides Petri et LTL.
Simulation comparée	<code>PetriNetIntegrationDemo.scala</code> , scénario Akka et correspondance Petri.
Lien GitHub	Présent dans le README et sur la page de garde de ce rapport.

11 Limites et dette technique

Le projet réalise les objectifs demandés, mais l'analyse détaillée fait apparaître plusieurs limites. Elles ne remettent pas en cause les propriétés vérifiées dans le périmètre actuel ; elles indiquent plutôt les points à renforcer pour un système plus réaliste.

Limite	Impact	Recommandation
Soldes abstraits dans le réseau de Pétri	Moyen	représenter les montants avec des réseaux colorés ou une abstraction plus fine des seuils de solde
Absence de timeout applicatif sur un virement en attente	Élevé	ajouter un timer côté source pour annuler et rembourser si la destination ne répond pas
Cache des soldes maintenu par Banque	Moyen	considérer le compte comme source de vérité et recalculer le total par consultation ou événement fiable
Réseau complet généré pour un nombre fixe de comptes	Moyen	ajouter un modèle paramétrable ou une preuve par schéma pour un nombre quelconque de comptes
Pas de persistance des transactions	Faible à moyen	journaliser les opérations et rejouer l'état après redémarrage
Model checker LTL sans optimisations avancées	Faible	ajouter des réductions d'état, des BDD ou des contre-exemples plus compacts pour de grands réseaux

Les extensions possibles sont la visualisation graphique du réseau, l'export PNML, l'ajout de réseaux temporisés, une comparaison plus systématique entre traces Akka et traces Pétri, ou encore l'intégration directe des résultats LTL dans l'interface de démonstration.

12 Synthèse de vérification

La synthèse suivante reprend le format du rapport de référence : elle distingue les propriétés totalement vérifiées, les propriétés partiellement couvertes et les points restants à améliorer.

Propriété vérifiée	Résultat
Protocoles acteurs typés par ActorRef [T] et traits scellés	PASS
Routage banque vers comptes existants	PASS
Refus des comptes dupliqués ou créés avec solde négatif	PASS
Dépôt positif et retrait avec solde suffisant	PASS
Solde jamais négatif dans les tests et le modèle abstrait	PASS
Virement confirmé entre deux comptes actifs	PASS
Remboursement de la source si la destination refuse	PASS
Fermeture refusée pendant un virement en attente	PASS
Absence de deadlock dans les réseaux fournis	PASS
Vivacité et libération des verrous dans le modèle Pétri/LTL	PASS
Cohérence exacte des montants dans le réseau de Pétri	PARTIEL
Tolérance à une destination qui ne répond jamais	À AMÉLIORER

13 Conclusion

Le projet aboutit à une application bancaire distribuée complète, vérifiée par deux approches complémentaires. Akka/Scala fournit une simulation réaliste et testable, tandis que le réseau de Pétri et LTL donnent une analyse formelle des comportements critiques. Les propriétés principales attendues sont couvertes : absence de deadlock, bornitude, vivacité, invariants non négatifs et cohérence des virements.

Cette combinaison permet de relier clairement les exigences métier, l'implémentation distribuée et les garanties formelles demandées dans le cahier des charges.

Références

- [1] Christel Baier et Joost-Pieter Katoen, *Principles of Model Checking*, MIT Press, 2008.
- [2] Wolfgang Reisig, *Understanding Petri Nets*, Springer, 2013.
- [3] E. M. Clarke, O. Grumberg et D. Peled, *Model Checking*, MIT Press, 1999.
- [4] Leslie Lamport, "Proving the Correctness of Multiprocess Programs", *IEEE Transactions on Software Engineering*, 1977.
- [5] Amir Pnueli, "The Temporal Logic of Programs", *FOCS*, 1977.
- [6] Akka Documentation, <https://doc.akka.io/docs/akka/current/typed/>
- [7] Scala Documentation, <https://docs.scala-lang.org/>